To put it loosely, state is somebody else's problem. This also applies for state about the client; instead of keeping state about the client inside your application, it should be transferred to your application by the client.

Concurrency?

One problem with shared state is the trouble it creates for concurrent systems. If state depends heavily on things happening in a particular order, then a system where many things are happening at once is a challenge. CoffeeScript programs run on an event loop and only ever do one thing at a time, so the concurrency challenges aren't the same as in a threaded environment. That said, state is still a problem.

Function composition and functional programming in general don't work well when programs have explicit mutable state contained in variables or objects. You've learned that in order to be effective with function composition, you should avoid state in variables and objects wherever possible. When you do avoid state, you can glue functions together explicitly and also create abstractions. Functions can glue other functions together.

6.3. Abstraction

Abstractions can be created from an existing program by carving off small pieces and improving those—removing duplication as you go. In this section you'll continue to improve listing 6.2 by creating abstractions and removing duplication.

6.3.1. Extracting common code

In listing 6.2 much of the duplicated code should have either never happened or been refactored once it became apparent. How does it become apparent? Well, consider what the program actually does.